PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL)

10 Case Studies — Complete Report

Student Name:

_____

Roll Number:

_____

Branch / Section:

_____

College:

_____

This report contains the problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output panels, and conclusion for each case study.

How to use this report

The programs use beginner-level Python. Rates and percentages are demonstration assumptions; replace them if your faculty gave different rules. The dark output panels are sample console outputs, not screenshots captured from your own computer. If your lecturer requires real screenshots, run each program and insert screenshots from your Python editor.

Contents

1. Progressive Electricity Billing

2. Employee Payroll and Deduction Analysis

3. ATM Transaction Sequence Simulation

4. Warehouse Inventory Reconciliation

5. Smart Parking Fee Management

6. Password Policy Validation Engine

7. String-Based Examination Answer Evaluation

8. ATM Menu-Driven Banking Simulation

9. Food Waste Monitoring System

10. Library Borrowing and Fine Reconciliation

1. Progressive Electricity Billing

1. Problem Statement

Write a Python program to calculate a household electricity bill using progressive consumption slabs.

2. Objectives

To calculate the bill by applying different rates to different ranges of electricity units.

3. Assumptions

First 100 units cost Rs.2 per unit.

Units 101–200 cost Rs.3 per unit.

Units above 200 cost Rs.5 per unit.

No fixed charges, surcharge, or rebate is included.

4. Algorithm

1. Start the program.

2. Read the number of units consumed.

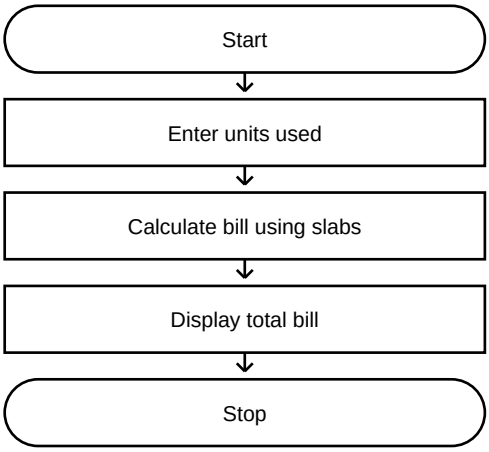
3. If units are at most 100, multiply all units by Rs.2.

4. If units are 101–200, charge the first 100 units at Rs.2 and the remaining units at Rs.3.

5. If units exceed 200, charge the first 100 at Rs.2, the next 100 at Rs.3, and the remaining units at Rs.5.

6. Display the bill and stop.

5. Flowchart



6. Pseudocode

START

INPUT units

IF units < 0

PRINT invalid units

ELSE IF units <= 100

bill = units * 2

ELSE IF units <= 200

bill = 100 * 2 + (units - 100) * 3

ELSE

bill = 100 * 2 + 100 * 3 + (units - 200) * 5

END IF

PRINT bill

STOP

7. Python Source Code

```
units = int(input("Enter electricity units: "))
if units < 0:
print("Invalid units")
elif units <= 100:
bill = units * 2
elif units <= 200:
bill = 100 * 2 + (units - 100) * 3
else:
bill = 100 * 2 + 100 * 3 + (units - 200) * 5
if units >= 0:
print("Electricity Bill =", bill)
```

8. Test Cases and Outputs

Units

Expected bill

50

Rs.100

150

Rs.350

250

Rs.850

Sample Output Screenshot (console-style panel)

Enter electricity units: 150

Electricity Bill = 350

9. Conclusion

The program uses if-elif-else statements and arithmetic to calculate an electricity bill using progressive slabs.

2. Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate an employee's gross salary and net salary after allowances and deductions.

2. Objectives

To prepare a simple salary statement using arithmetic operations.

3. Assumptions

Allowance is 20% of basic salary.

Deduction is 10% of basic salary.

Overtime and incentives are not included in this beginner version.

4. Algorithm

1. Start.

2. Read basic salary.

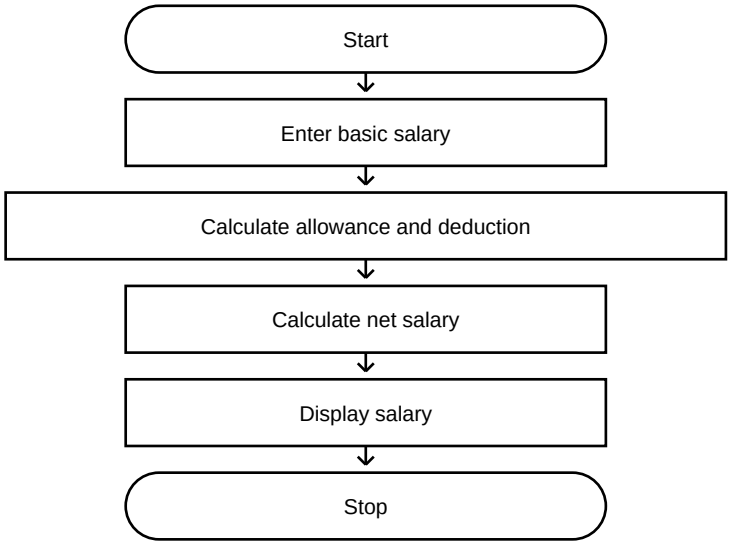
3. Calculate allowance as 20% of basic salary.

4. Calculate deduction as 10% of basic salary.

5. Calculate gross salary and net salary.

6. Display the salary details and stop.

5. Flowchart



6. Pseudocode

START

INPUT basic salary

allowance = basic salary * 0.20

deduction = basic salary * 0.10

gross = basic salary + allowance

net = gross - deduction

PRINT allowance, deduction, gross, net

STOP

7. Python Source Code

```
basic = float(input("Enter basic salary: "))
if basic < 0:
print("Invalid salary")
else:
allowance = basic * 0.20
deduction = basic * 0.10
gross = basic + allowance
net = gross - deduction
print("Basic Salary =", basic)
print("Allowance =", allowance)
print("Deduction =", deduction)
print("Gross Salary =", gross)
print("Net Salary =", net)
```

8. Test Cases and Outputs

Basic salary

Expected net salary

10000

Rs.11000

20000

Rs.22000

30000

Rs.33000

Sample Output Screenshot (console-style panel)

Enter basic salary: 20000

Basic Salary = 20000.0

Allowance = 4000.0

Deduction = 2000.0

Gross Salary = 24000.0

Net Salary = 22000.0

9. Conclusion

The program calculates allowance, deduction, gross salary, and net salary using simple formulas.

3. ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of deposits and withdrawals while maintaining a bank balance.

2. Objectives

To practise loops, conditions, and balance updates for multiple transactions.

3. Assumptions

The opening balance is Rs.1,000.

The user can deposit, withdraw, check balance, or exit.

A withdrawal is allowed only if the amount is positive and does not exceed the balance.

4. Algorithm

1. Start with a balance of Rs.1,000.

2. Display the transaction menu.

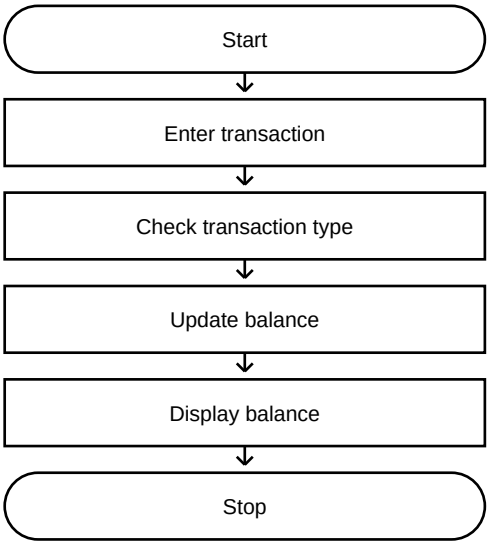
3. Read the user's choice.

4. Perform deposit, withdrawal, or balance check.

5. Reject invalid amounts or insufficient balance.

6. Repeat until Exit is selected.

5. Flowchart



6. Pseudocode

START

balance = 1000

REPEAT

DISPLAY menu

INPUT choice

IF choice = 1: INPUT amount; IF amount > 0, add to balance

ELSE IF choice = 2: INPUT amount; withdraw if valid and affordable

ELSE IF choice = 3: PRINT balance

ELSE IF choice = 4: EXIT loop

ELSE: PRINT invalid choice

UNTIL choice = 4

STOP

7. Python Source Code

```
balance = 1000
while True:
print("\n1. Deposit")
print("2. Withdraw")
print("3. Check Balance")
print("4. Exit")
choice = int(input("Enter choice: "))
if choice == 1:
amount = float(input("Enter deposit amount: "))
if amount > 0:
balance += amount
print("Deposit successful")
else:
print("Invalid amount")
elif choice == 2:
amount = float(input("Enter withdrawal amount: "))
if 0 < amount <= balance:
balance -= amount
print("Withdrawal successful")
else:
print("Invalid amount or insufficient balance")
elif choice == 3:
print("Balance =", balance)
elif choice == 4:
break
else:
print("Invalid choice")
print("Thank you")
```

8. Test Cases and Outputs

Transactions

Expected final balance

Deposit Rs.500

Rs.1500

Withdraw Rs.200

Rs.800

Withdraw Rs.1500 from Rs.1000

Rejected; balance stays Rs.1000

Sample Output Screenshot (console-style panel)

1. Deposit

2. Withdraw

3. Check Balance

4. Exit

Enter choice: 1

Enter deposit amount: 500

Deposit successful

Enter choice: 3

Balance = 1500.0

Enter choice: 4

Thank you

9. Conclusion

The program simulates basic ATM transactions using a while loop and conditional statements.

4. Warehouse Inventory Reconciliation

1. Problem Statement

Calculate closing stock using opening stock, incoming stock, and dispatched stock, and identify shortages.

2. Objectives

To maintain simple stock records and detect when dispatches exceed available stock.

3. Assumptions

All stock quantities must be non-negative.

Closing stock equals opening stock plus incoming stock minus dispatched stock.

A negative closing stock indicates a shortage.

4. Algorithm

1. Read the number of products.

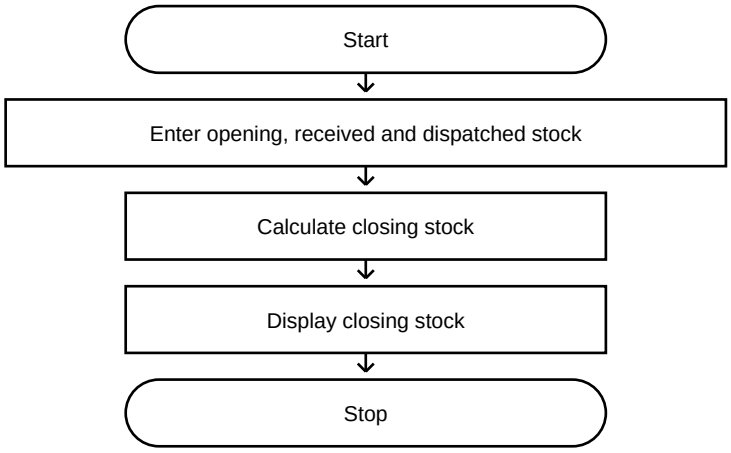
2. For each product, read its name and stock quantities.

3. Validate that all quantities are non-negative.

4. Calculate closing stock.

5. Display closing stock and report a shortage if it is negative.

5. Flowchart



6. Pseudocode

START

INPUT number of products

FOR each product

INPUT name, opening, incoming, dispatched

IF any quantity is negative

PRINT invalid stock details

ELSE

closing = opening + incoming - dispatched

PRINT closing

IF closing < 0: PRINT shortage

ELSE: PRINT stock available

END IF

END FOR

STOP

7. Python Source Code

```
n = int(input("Enter number of products: "))
for i in range(n):
print("\nProduct", i + 1)
name = input("Enter product name: ")
opening = int(input("Enter opening stock: "))
incoming = int(input("Enter incoming stock: "))
dispatched = int(input("Enter dispatched stock: "))
if opening < 0 or incoming < 0 or dispatched < 0:
print("Invalid stock details")
else:
closing = opening + incoming - dispatched
print("Product Name:", name)
print("Closing Stock:", closing)
if closing < 0:
print("Stock shortage detected!")
else:
print("Stock available")
```

8. Test Cases and Outputs

Opening, incoming, dispatched

Expected result

100, 50, 30

Closing stock = 120

40, 10, 60

Shortage of 10

20, 0, 10

Closing stock = 10

Sample Output Screenshot (console-style panel)

Enter number of products: 1

Product 1

Enter product name: Pen

Enter opening stock: 100

Enter incoming stock: 50

Enter dispatched stock: 30

Product Name: Pen

Closing Stock: 120

Stock available

9. Conclusion

The program calculates stock balances and flags shortages using variables, loops, arithmetic, and conditions.

5. Smart Parking Fee Management

1. Problem Statement

Calculate a parking fee based on vehicle type and the number of parking hours.

2. Objectives

To calculate parking charges for cars and bikes and validate input.

3. Assumptions

Cars cost Rs.30 per hour.

Bikes cost Rs.15 per hour.

Any started/partial hour is not rounded up; the entered integer is treated as total chargeable hours.

Parking hours must be greater than zero.

4. Algorithm

1. Read vehicle type and parking hours.

2. Check that the hours are positive.

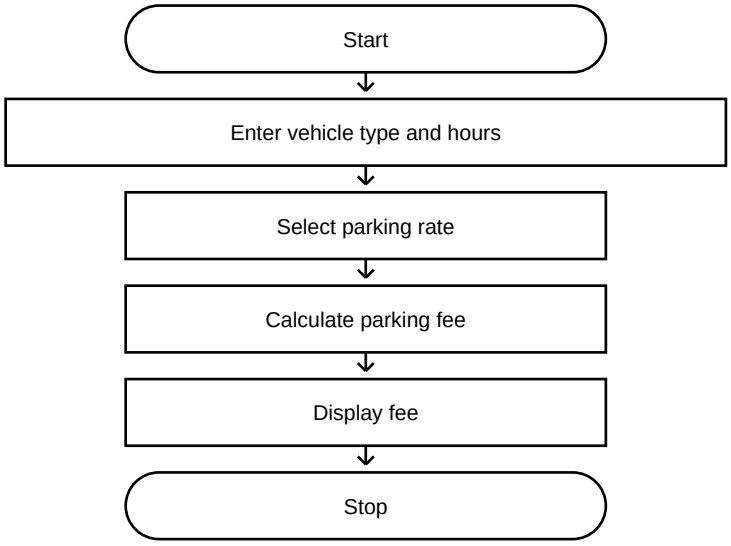
3. If the vehicle is a car, multiply hours by Rs.30.

4. If the vehicle is a bike, multiply hours by Rs.15.

5. For any other vehicle type, show an error.

6. Display the fee.

5. Flowchart



6. Pseudocode

START

INPUT vehicle, hours

IF hours <= 0

PRINT invalid parking hours

ELSE IF vehicle = car

fee = hours * 30

ELSE IF vehicle = bike

fee = hours * 15

ELSE

PRINT invalid vehicle type

END IF

PRINT fee when valid

STOP

7. Python Source Code

```
vehicle = input("Enter vehicle type (car/bike): ").lower()
hours = int(input("Enter parking hours: "))
if hours <= 0:
print("Invalid parking hours")
elif vehicle == "car":
print("Parking Fee =", hours * 30)
elif vehicle == "bike":
print("Parking Fee =", hours * 15)
else:
print("Invalid vehicle type")
```

8. Test Cases and Outputs

Vehicle, hours

Expected fee

car, 2

Rs.60

bike, 3

Rs.45

car, 0

Invalid parking hours

Sample Output Screenshot (console-style panel)

Enter vehicle type (car/bike): car

Enter parking hours: 2

Parking Fee = 60

9. Conclusion

The program calculates parking fees based on vehicle category and hours using conditions and multiplication.

6. Password Policy Validation Engine

1. Problem Statement

Check whether a password satisfies basic length, digit, and uppercase-letter rules.

2. Objectives

To validate a sample password using string operations and conditions.

3. Assumptions

The password must contain at least 8 characters.

It must contain at least one digit.

It must contain at least one uppercase letter.

This is a basic demonstration and is not a complete security check.

4. Algorithm

1. Read a password.

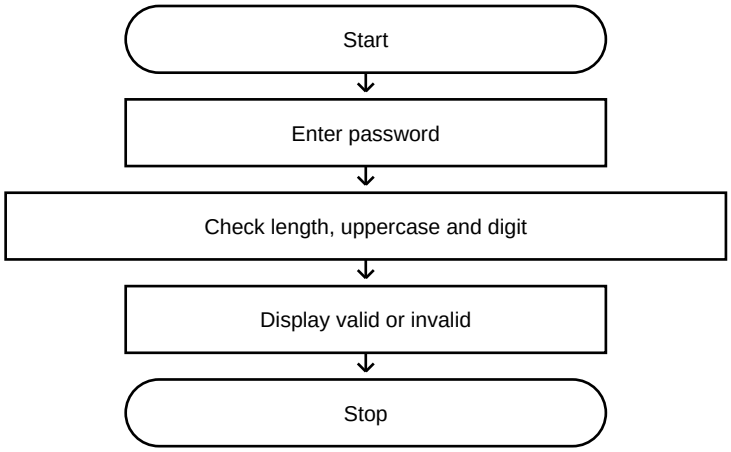
2. Check whether its length is at least 8.

3. Check whether it contains a digit.

4. Check whether it contains an uppercase letter.

5. Display the first failed rule or report that the password is valid.

5. Flowchart



6. Pseudocode

START

INPUT password

IF length(password) < 8

PRINT minimum length error

ELSE IF password has no digit

PRINT number required

ELSE IF password has no uppercase letter

PRINT uppercase letter required

ELSE

PRINT password is valid

END IF

STOP

7. Python Source Code

```
password = input("Enter password: ")
if len(password) < 8:
print("Password must contain at least 8 characters")
elif not any(ch.isdigit() for ch in password):
print("Password must contain a number")
elif not any(ch.isupper() for ch in password):
print("Password must contain an uppercase letter")
else:
print("Password is valid")
```

8. Test Cases and Outputs

Password

Expected result

Hello123

Valid

abc12

Minimum length error

helloworld

Number required

hello1234

Uppercase letter required

Sample Output Screenshot (console-style panel)

Enter password: Hello123

Password is valid

9. Conclusion

The program uses len(), string character checks, and if-elif-else statements to validate a password.

7. String-Based Examination Answer Evaluation

1. Problem Statement

Compare a student's answers with an answer key and calculate correct and wrong answers.

2. Objectives

To practise strings, loops, indexing, and counting correct answers.

3. Assumptions

The answer key is ABCDE.

The student must enter exactly five answers.

Each correct answer earns one mark; there is no negative marking.

Answers are compared without considering letter case.

4. Algorithm

1. Store the answer key.

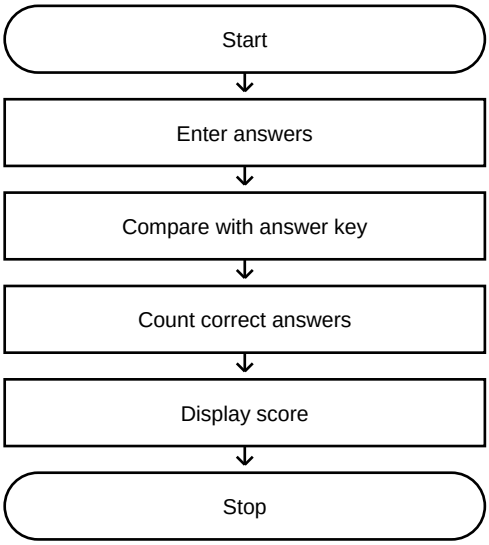
2. Read the student's answers and convert them to uppercase.

3. Check that exactly five answers are entered.

4. Compare each answer with the corresponding key character.

5. Count correct and wrong answers and display the result.

5. Flowchart



6. Pseudocode

START

key = ABCDE

INPUT answer

CONVERT answer to uppercase

IF length(answer) != 5

PRINT enter exactly 5 answers

ELSE

score = 0

FOR each index from 0 to 4

IF answer[index] = key[index]: score = score + 1

END FOR

PRINT correct = score

PRINT wrong = 5 - score

END IF

STOP

7. Python Source Code

```
key = "ABCDE"
answer = input("Enter 5 answers (A-E): ").upper()
if len(answer) != len(key):
print("Enter exactly 5 answers")
else:
score = 0
for i in range(len(key)):
if answer[i] == key[i]:
score += 1
print("Correct Answers =", score)
print("Wrong Answers =", len(key) - score)
```

8. Test Cases and Outputs

Answers

Expected result

ABCDE

5 correct, 0 wrong

ABCDA

4 correct, 1 wrong

AAAAA

1 correct, 4 wrong

ABC

Length error

Sample Output Screenshot (console-style panel)

Enter 5 answers (A-E): ABCDA

Correct Answers = 4

Wrong Answers = 1

9. Conclusion

The program compares strings character by character and counts correct and incorrect answers.

8. ATM Menu-Driven Banking Simulation

1. Problem Statement

Create a simple ATM menu that lets a user check balance, deposit money, withdraw money, or exit.

2. Objectives

To practise menu-driven programming with if-elif-else statements.

3. Assumptions

The opening balance is Rs.5,000.

Only one menu action is performed before the program ends.

Deposits and withdrawals must be positive.

Withdrawals cannot exceed the available balance.

4. Algorithm

1. Set the opening balance to Rs.5,000.

2. Display the menu and read a choice.

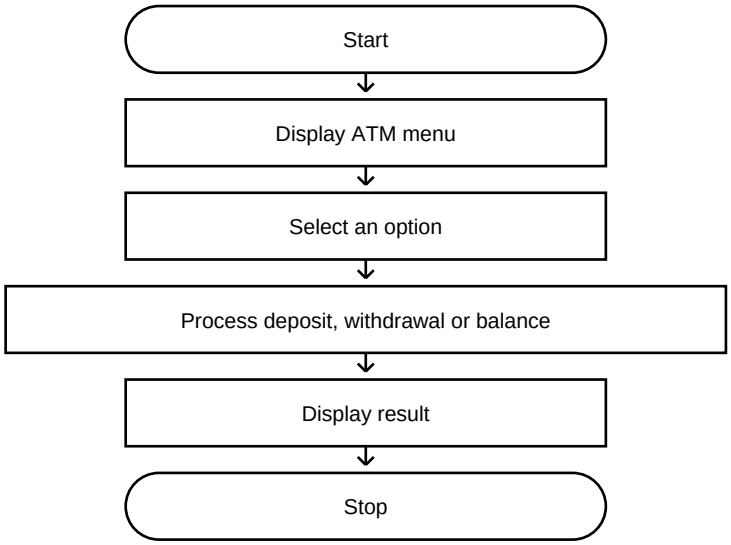
3. Perform the selected operation.

4. Validate deposit or withdrawal amounts.

5. Display the updated balance where relevant.

6. Exit after the selected action.

5. Flowchart



6. Pseudocode

START

balance = 5000

DISPLAY menu

INPUT choice

IF choice = 1: PRINT balance

ELSE IF choice = 2: INPUT amount; add if positive

ELSE IF choice = 3: INPUT amount; subtract if valid and affordable

ELSE IF choice = 4: PRINT thank you

ELSE: PRINT invalid choice

STOP

7. Python Source Code

```
balance = 5000
print("1. Check Balance")
print("2. Deposit")
print("3. Withdraw")
print("4. Exit")
choice = int(input("Enter choice: "))
if choice == 1:
print("Balance =", balance)
elif choice == 2:
amount = float(input("Enter deposit amount: "))
if amount > 0:
balance += amount
print("New Balance =", balance)
else:
print("Invalid amount")
elif choice == 3:
amount = float(input("Enter withdrawal amount: "))
if 0 < amount <= balance:
balance -= amount
print("New Balance =", balance)
else:
print("Invalid amount or insufficient balance")
elif choice == 4:
print("Thank you")
else:
print("Invalid choice")
```

8. Test Cases and Outputs

Choice and amount

Expected result

1

Balance = Rs.5000

2, deposit 500

New balance = Rs.5500

3, withdraw 6000

Rejected

4

Thank you

Sample Output Screenshot (console-style panel)

1. Check Balance

2. Deposit

3. Withdraw

4. Exit

Enter choice: 2

Enter deposit amount: 500

New Balance = 5500.0

9. Conclusion

The program performs one banking operation using a menu, conditions, and balance calculations.

9. Food Waste Monitoring System

1. Problem Statement

Calculate leftover meals and the percentage of prepared meals that were wasted.

2. Objectives

To measure food waste and calculate meals consumed using basic arithmetic.

3. Assumptions

The number of meals prepared must be greater than zero.

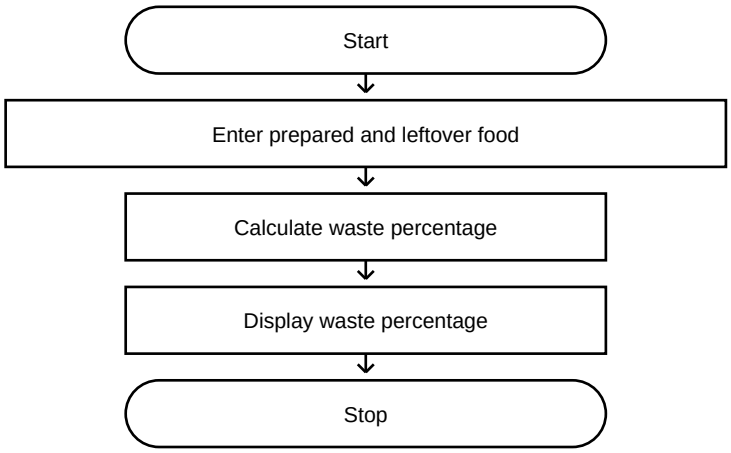
Leftover meals must be between zero and the number prepared.

All leftover meals are treated as wasted meals.

4. Algorithm

1. Read meals prepared and leftover meals.
2. Validate the values.
3. Calculate waste percentage.
4. Calculate meals consumed.
5. Display waste and consumption details.

5. Flowchart



6. Pseudocode

START

INPUT prepared, leftover

IF prepared <= 0 OR leftover < 0 OR leftover > prepared

PRINT invalid meal details

ELSE

percentage = leftover / prepared * 100

consumed = prepared - leftover

PRINT leftover, percentage, consumed

END IF

STOP

7. Python Source Code

```
prepared = int(input("Enter meals prepared: "))
leftover = int(input("Enter leftover meals: "))
if prepared <= 0 or leftover < 0 or leftover > prepared:
print("Invalid meal details")
else:
percentage = leftover / prepared * 100
consumed = prepared - leftover
print("Meals Wasted =", leftover)
print("Waste Percentage =", percentage, "%")
print("Meals Consumed =", consumed)
```

8. Test Cases and Outputs

Prepared, leftover

Expected result

100, 20

20% wasted; 80 consumed

50, 0

0% wasted; 50 consumed

30, 35

Invalid meal details

Sample Output Screenshot (console-style panel)

Enter meals prepared: 100

Enter leftover meals: 20

Meals Wasted = 20

Waste Percentage = 20.0 %

Meals Consumed = 80

9. Conclusion

The program calculates food waste percentage and meals consumed using arithmetic and input validation.

10. Library Borrowing and Fine Reconciliation

1. Problem Statement

Calculate a library fine based on the number of days a book is kept beyond the allowed borrowing period.

2. Objectives

To use tuples and conditional statements to store borrowing details and calculate a fine.

3. Assumptions

The allowed borrowing period is 7 days.

The fine is Rs.5 for each day beyond 7 days.

No fine is charged when the book is returned within 7 days.

4. Algorithm

1. Read borrower name, book name, and days kept.

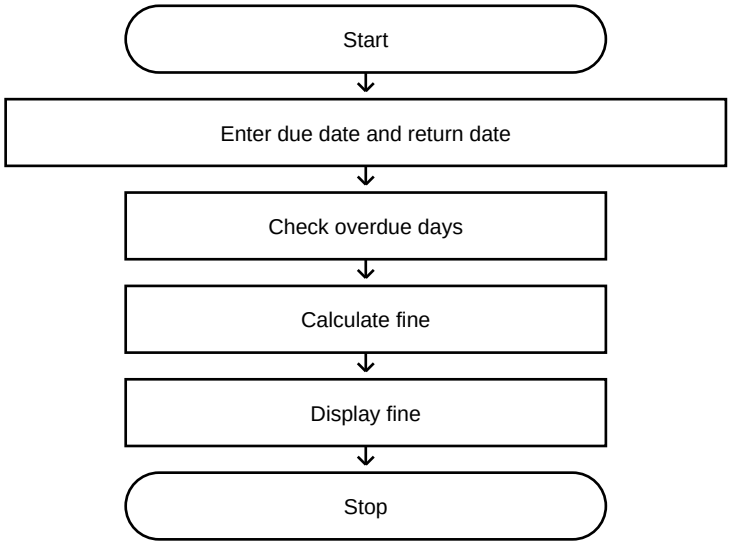
2. Store the details in a tuple.

3. If days kept exceed 7, calculate the fine.

4. Otherwise, set the fine to zero.

5. Display borrower, book, days, and fine.

5. Flowchart



6. Pseudocode

START

INPUT borrower, book, days

record = (borrower, book, days)

IF days < 0

PRINT invalid days

ELSE IF days > 7

fine = (days - 7) * 5

ELSE

fine = 0

END IF

PRINT record and fine

STOP

7. Python Source Code

```
borrower = input("Enter student name: ")
book = input("Enter book name: ")
days = int(input("Enter days kept: "))
if days < 0:
print("Invalid number of days")
else:
record = (borrower, book, days)
if days > 7:
fine = (days - 7) * 5
else:
fine = 0
print("Borrower =", record[0])
print("Book =", record[1])
print("Days Kept =", record[2])
print("Fine = Rs.", fine)
```

8. Test Cases and Outputs

Days kept

Expected fine

5

Rs.0

7

Rs.0

10

Rs.15

-1

Invalid number of days

Sample Output Screenshot (console-style panel)

Enter student name: Ravi

Enter book name: Python

Enter days kept: 10

Borrower = Ravi

Book = Python

Days Kept = 10

Fine = Rs. 15

9. Conclusion

The program stores borrowing details in a tuple and calculates overdue fines using conditional statements.